

Tracing Heap, Graph, dan Edge Case pada Hospital Patient Routing

Nama Anggota:

Adi Satria Pangestu	5027231043
Muhammad Rafi Pramudya Putra	5027251024
Daffa Ulhaq Fadhlurrahman	5027251033
Muhammad Nadhif Pasya Ikhsan	5027251084
Michiko Artika Satriyo	5027251105

1. Tracing Heap

Disini kami mengedit code dan menambahkan debugger untuk memudahkan melihat proses dalam Heap. Ada beberapa function yang kita fokuskan untuk tracing pertama pada fungsi insert yang melakukan heapify Up

```
public void insert(Pasien p) {
    heap.add(p);
    int i = heap.size() - 1;

    System.out.println(" [INSERT] " + p.nama + " (Prio: " + p.prioritas + ", Masuk: " + p.waktuMasuk + ") ditempatkan di indeks [" + i + "]");

    while (i > 0) {
        int parent = (i - 1) / 2;
        if (LebihUtama(heap.get(i), heap.get(parent))) {
            System.out.println(" --> HEAPIFY-UP: " + heap.get(i).nama + " [indeks " + i + "] LEBIH UTAMA dari " +
                               heap.get(parent).nama + " [indeks " + parent + "]. Tukar posisi!");
            swap(i, parent);
            i = parent;
        } else {
            break;
        }
    }
}
```

Lalu ada function Heapify Down yang terjadi ketika node root diambil maka akan terjadi kekosongan root, sehingga heap akan mengambil node paling rendah memindahkan sementara di root dan akan melakukan Heapify Down

```

private void heapifyDown(int i) {
    int n = heap.size();
    while (true) {
        int kiri = 2 * i + 1, kanan = 2 * i + 2, terbesar = i;

        if (kiri < n && LebihUtama(heap.get(kiri), heap.get(terbesar)))
            terbesar = kiri;
        if (kanan < n && LebihUtama(heap.get(kanan), heap.get(terbesar)))
            terbesar = kanan;

        if (terbesar == i) {
            System.out.println("    [HEAPIFY-DOWN] selesai. Posisi " + heap.get(i).nama + " sudah pas di indeks [" + i + "].");
            break;
        }
        System.out.println("    [HEAPIFY-DOWN] " + heap.get(terbesar).nama + " di indeks [" + terbesar + "] menyalip " +
            heap.get(i).nama + " di indeks [" + i + "]. Tukar posisi");
        swap(i, terbesar);
        i = terbesar;
    }
}

```

Untuk menguji nya kita akan menggunakan script dengan mencoba memasukan 3 pasien normal dan memasukan 1 pasien dengan kategori darurat untuk memperlihatkan proses Heapifynya

```

package src;

import src.heap.MaxHeapAntrian;
import src.model.Pasien;
import java.util.List;

public class TestMaxHeap {
    public static void main(String[] args) {
        MaxHeapAntrian antrianRS = new MaxHeapAntrian();

        System.out.println(x: "    TRACING MAXHEAP DENGAN MEMASUKAN PASIEN    ");

        System.out.println(x: "\n[Kasus 1] Memasukkan 3 Pasien Normal");
        antrianRS.insert(new Pasien(id: 101, nama: "Pasien_A_Normal", prioritas: 2, waktuMasuk: 5));
        antrianRS.insert(new Pasien(id: 102, nama: "Pasien_B_Normal", prioritas: 2, waktuMasuk: 8));
        antrianRS.insert(new Pasien(id: 103, nama: "Pasien_C_Normal", prioritas: 2, waktuMasuk: 10));

        System.out.println(x: "\n[Kasus 2] Datang Pasien Baru Kategori Darurat");
        antrianRS.insert(new Pasien(id: 104, nama: "Pasien_D_DARURAT", prioritas: 1, waktuMasuk: 12));

        while (antrianRS.size() > 0) {
            antrianRS.extractMax();
        }
    }
}

```

```

TRACING MAXHEAP DENGAN MEMASUKAN PASIEN

[Kasus 1] Memasukkan 3 Pasien Normal
[INSERT] Pasien_A_Normal (Prio: 2, Masuk: 5) ditempatkan di indeks [0]
[INSERT] Pasien_B_Normal (Prio: 2, Masuk: 8) ditempatkan di indeks [1]
[INSERT] Pasien_C_Normal (Prio: 2, Masuk: 10) ditempatkan di indeks [2]

[Kasus 2] Datang Pasien Baru Kategori Darurat
[INSERT] Pasien_D_DARURAT (Prio: 1, Masuk: 12) ditempatkan di indeks [3]
--> HEAPIFY-UP: Pasien_D_DARURAT [indeks 3] LEBIH UTAMA dari Pasien_B_Normal [indeks 1]. Tukar posisi!
--> HEAPIFY-UP: Pasien_D_DARURAT [indeks 1] LEBIH UTAMA dari Pasien_A_Normal [indeks 0]. Tukar posisi!

[EXTRACT] Melayani pasien teratas: Pasien_D_DARURAT
[REPLACE] Memindahkan elemen terakhir (Pasien_B_Normal) sementara ke indeks [0]
[HEAPIFY-DOWN] Pasien_A_Normal di indeks [1] menyalip Pasien_B_Normal di indeks [0]. Tukar posisi
[HEAPIFY-DOWN] selesai. Posisi Pasien_B_Normal sudah pas di indeks [1].

[EXTRACT] Melayani pasien teratas: Pasien_A_Normal
[REPLACE] Memindahkan elemen terakhir (Pasien_C_Normal) sementara ke indeks [0]
[HEAPIFY-DOWN] Pasien_B_Normal di indeks [1] menyalip Pasien_C_Normal di indeks [0]. Tukar posisi
[HEAPIFY-DOWN] selesai. Posisi Pasien_C_Normal sudah pas di indeks [1].

[EXTRACT] Melayani pasien teratas: Pasien_B_Normal
[REPLACE] Memindahkan elemen terakhir (Pasien_C_Normal) sementara ke indeks [0]
[HEAPIFY-DOWN] selesai. Posisi Pasien_C_Normal sudah pas di indeks [0].

[EXTRACT] Melayani pasien teratas: Pasien_C_Normal

```

Disini terlihat Proses mulai dari insert pasien A sampai pasien C , lalu ketika ada pasien D masuk dengan kategori darurat Heap langsung melakukan Heapify up dan menempatkan Pasien darurat tersebut ke root agar segera ditangani. lalu diakhir code kita melakukan pengambilan node satu persatu mulai dari node root hingga bawah dengan cara mengambil

node root dan memindahkan node paling bawah menjadi root lalu melakukan Heapify Down. Proses tersebut terus diulang sampai node habis

2. Tracing Graph

untuk men trace graph kita fokuskan pada Bfs dan Dijkstra nya , sama seperti sebelumnya kami membuat debugger untuk melihat proses setiap fungsinya. Kami juga membuat script testing untuk menambahkan node ruangan dan menghubungkan node tersebut dan mencoba mengugnkana fungsi bfs dan dijkstranya

```
public class TestGraph {
    Run | Debug
    public static void main(String[] args) {
        Graph rsMap = new Graph();

        rsMap.tambahRuangan(new Ruangan(id: 0, nama: "Ruang Tunggu", kategori: "UTAMA", subKategori: null, rataRataLayana..0, 99, 1));
        rsMap.tambahRuangan(new Ruangan(id: 1, nama: "IGD 1", kategori: "IGD", subKategori: null, rataRataLayana..5, 2, 1));
        rsMap.tambahRuangan(new Ruangan(id: 2, nama: "Poli Umum", kategori: "POLI", subKategori: "UMUM", rataRataLayana..10, 3, 1));
        rsMap.tambahRuangan(new Ruangan(id: 3, nama: "Lab Darah", kategori: "LAB", subKategori: "DARAH", rataRataLayana..7, 1, 2));

        rsMap.tambahEdge(a: 0, b: 1, waktu: 2);
        rsMap.tambahEdge(a: 0, b: 2, waktu: 8);
        rsMap.tambahEdge(a: 1, b: 3, waktu: 3);
        rsMap.tambahEdge(a: 2, b: 3, waktu: 1);

        System.out.println(x: "          SIMULASI TRACING MANUAL PROSES GRAPH (BFS & DIJKSTRA)          ");

        ListHasilBFS> kandidatLab = rsMap.bfsCarikategori(start: 0, kategori: "LAB", subKategori: null);
        HasilDijkstra ruteTercepat = rsMap.dijkstra(start: 0);

        List<Integer> jalurKelab = rsMap.jalurDijkstra(ruteTercepat.prev, start: 0, tujuan: 3);

        System.out.print(s: "Rute terbaik ke Lab Darah: ");
        for(int i = 0; i < jalurKelab.size(); i++) {
            System.out.print(rsMap.nodes.get(jalurKelab.get(i)).nama + (i < jalurKelab.size() - 1 ? " -> " : ""));
        }
        System.out.println("\nTotal Waktu Tempuh Tercepat: " + ruteTercepat.dist[3] + " Detik");
    }
}
```

```
SIMULASI TRACING MANUAL PROSES GRAPH (BFS & DIJKSTRA)

--- START BFS TRACING (Mencari Kategori: LAB) ---
[BFS POLL] Mengecek Node #0 (Ruang Tunggu) | Queue Sisa: []
-> Tetangga #1 (IGD 1) belum dikunjungi. Masuk Queue.
-> Tetangga #2 (Poli Umum) belum dikunjungi. Masuk Queue.
[BFS POLL] Mengecek Node #1 (IGD 1) | Queue Sisa: [2]
-> Tetangga #3 (Lab Darah) belum dikunjungi. Masuk Queue.
[BFS POLL] Mengecek Node #2 (Poli Umum) | Queue Sisa: [3]
[BFS POLL] Mengecek Node #3 (Lab Darah) | Queue Sisa: []
>> [FOUND] Cocok! Menemukan ruangan target: Lab Darah

--- START DIJKSTRA TRACING (Mencari Rute Tercepat dari Ruang Tunggu) ---
[DIJKSTRA POP] Node #0 (Ruang Tunggu) tereksekusi dengan jarak sementara = 0
-> Cek jalur ke tetangga #1 (IGD 1) lewat #0 | Bobot lorong = 2
[RELAXATION] Jarak ke IGD 1 BERHASIL DIPERBAIKI! => 2
-> Cek jalur ke tetangga #2 (Poli Umum) lewat #0 | Bobot lorong = 8
[RELAXATION] Jarak ke Poli Umum BERHASIL DIPERBAIKI! => 8
[DIJKSTRA POP] Node #1 (IGD 1) tereksekusi dengan jarak sementara = 2
-> Cek jalur ke tetangga #0 (Ruang Tunggu) lewat #1 | Bobot lorong = 2
[SKIP] Jalur lewat #1 (4) tidak lebih cepat dari data saat ini (0)
-> Cek jalur ke tetangga #3 (Lab Darah) lewat #1 | Bobot lorong = 3
[RELAXATION] Jarak ke Lab Darah BERHASIL DIPERBAIKI! => 5
[DIJKSTRA POP] Node #3 (Lab Darah) tereksekusi dengan jarak sementara = 5
-> Cek jalur ke tetangga #1 (IGD 1) lewat #3 | Bobot lorong = 3
[SKIP] Jalur lewat #3 (8) tidak lebih cepat dari data saat ini (2)
-> Cek jalur ke tetangga #2 (Poli Umum) lewat #3 | Bobot lorong = 1
[RELAXATION] Jarak ke Poli Umum BERHASIL DIPERBAIKI! 8 -> 6
[DIJKSTRA POP] Node #2 (Poli Umum) tereksekusi dengan jarak sementara = 6
-> Cek jalur ke tetangga #0 (Ruang Tunggu) lewat #2 | Bobot lorong = 8
[SKIP] Jalur lewat #2 (14) tidak lebih cepat dari data saat ini (0)
-> Cek jalur ke tetangga #3 (Lab Darah) lewat #2 | Bobot lorong = 1
[SKIP] Jalur lewat #2 (7) tidak lebih cepat dari data saat ini (5)
[DIJKSTRA POP] Node #2 (Poli Umum) tereksekusi dengan jarak sementara = 8
Rute terbaik ke Lab Darah: Ruang Tunggu -> IGD 1 -> Lab Darah
Total Waktu Tempuh Tercepat: 5 Detik
```

Dari output tersebut terlihat untuk bfs menjelajahi tiap tetangga menggunakan sistem queue secara berurutan. Untuk Dijkstra melakukan cek node current lalu memasukan node tetangga ke dalam heap dan kembali mengecek node root heap sampai selesai. lalu hasilnya berupa sebuah list dengan waktu tercepat tiap node. program utama tinggal memanggil berapa waktu tempuhnya dan mengambil rute listnya

3.Tracing Edge Case

Disini kami merepresentasikan Edge case dengan sebuah kasus mencari jarak terpendek antara node yang saling tidak berhubungan dan mencoba memberi output jalur dari start menuju node tujuan

```
public class TestDisconnectedGraph {
    public static void main(String[] args) {
        Graph rsMap = new Graph();

        rsMap.tambahRuangan(new Ruangan(id: 0, nama: "Ruang Tunggu Utama", kategori: "UTAMA", subkategori: null, rataRataLayana: 0, 99, 1));
        rsMap.tambahRuangan(new Ruangan(id: 1, nama: "IGD 1", kategori: "IGD", subkategori: null, rataRataLayana: 5, 2, 1));
        rsMap.tambahEdge(a: 0, b: 1, waktu: 2);
        rsMap.tambahRuangan(new Ruangan(id: 2, nama: "Poli VIP Lantai 3", kategori: "POLI", subkategori: "VIP", rataRataLayana: 15, 1, 3));

        HasilDijkstra rute = rsMap.dijkstra(start: 0);
        int jarakKePoliVIP = rute.dist[2];
        System.out.println("Nilai dist[2] di dalam Array Jarak: " + jarakKePoliVIP);

        try {
            List<Integer> jalur = rsMap.jalurDijkstra(rute.prev, start: 0, tujuan: 2);
            System.out.print("Hasil cetak jalur: ");
            for (int id : jalur) {
                System.out.print(rsMap.nodes.get(id).nama + " -> ");
            }
            System.out.println("Selesai.");
        } catch (Exception e) {
            System.out.println("CRASH DETECTED Program error: " + e.getMessage());
        }
    }
}
```

```
--- START DIJKSTRA TRACING (Mencari Rute Tercepat dari Ruang Tunggu Utama) ---
[DIJKSTRA POP] Node #0 (Ruang Tunggu Utama) tereksekusi dengan jarak sementara = 0
-> Cek jalur ke tetangga #1 (IGD 1) lewat #0 | Bobot lorong = 2
[RELAXATION] Jarak ke IGD 1 BERHASIL DIPERBAIKI! => 2
[DIJKSTRA POP] Node #1 (IGD 1) tereksekusi dengan jarak sementara = 2
-> Cek jalur ke tetangga #0 (Ruang Tunggu Utama) lewat #1 | Bobot lorong = 2
[SKIP] Jalur lewat #1 (4) tidak lebih cepat dari data saat ini (0)
Nilai dist[2] di dalam Array Jarak: 2147483647
Hasil cetak jalur: Poli VIP Lantai 3 -> Selesai.
```

Terlihat dari output bahwa nilai jaraknya merupakan integer yang sangat panjang karena ketika dijkstra mencoba melakukan proses dari start algoritma tersebut tidak mencapai node tujuan sehingga , isi dari array dengan index[2] berisi nilai `MAX_VALUE` lalu untuk jalur pathnya akan terisi node tersebut dan langsung selesai dikarenakan ketika fungsi `JalurDijkstra` di proses nilai dari `prev[2]` bernilai -1 sehingga while loop langsung keluar dan node hasil hanya menyimpan node tersebut saja.